

Java Annotations

Definition

A Java Annotation is a special marker that provides [metadata \(information\)](#) about a program to the compiler, JVM, or other tools. It does not directly change the program's logic.

An annotation is a [special tag that provides additional information \(metadata\)](#) about a Java program to the compiler or JVM.

@Override

Indicates that a method overrides a method in its superclass

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {

    @Override
    void sound() {
        System.out.println("Dog barks");
    }

    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Dog barks

If you make a mistake in the method name, the compiler reports an error because [sounds\(\)](#) does not override any method

@Deprecated

Indicates that a class, method, or field is out-dated and should not be used.

The @Deprecated annotation warns developers not to use a method because it may be removed or replaced in the future:

```
public class Main {
    @Deprecated
    static void oldMethod() {
        System.out.println("This method is outdated.");
    }
}
```

```
public static void main(String[] args) {  
    oldMethod(); // This will show a warning in most IDEs  
}  
}
```

Even though the method runs and prints the message, most IDEs or compilers will show a warning like this

“warning: [deprecation] oldMethod() in Main has been deprecated”

@SuppressWarnings Annotation

Suppresses compiler warnings

```
import java.util.ArrayList;  
  
public class Demo {  
    @SuppressWarnings("unchecked")  
    public static void main(String[] args) {  
  
        ArrayList list = new ArrayList();  
  
        list.add("Java");  
        list.add(100);  
  
        System.out.println(list);  
    }  
}
```

[Java, 100]

Without @SuppressWarnings, the compiler may display warnings about raw types.